

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

Software Design Report

Project Name

Khoj

Submitted By:

Group 12

Talha Zubayer (230042119)
Ayman Rahman Bhuyian (230042140)
Md Nadid Hasan (230042146)

SWE 4404 : SPL II

Model Diagram

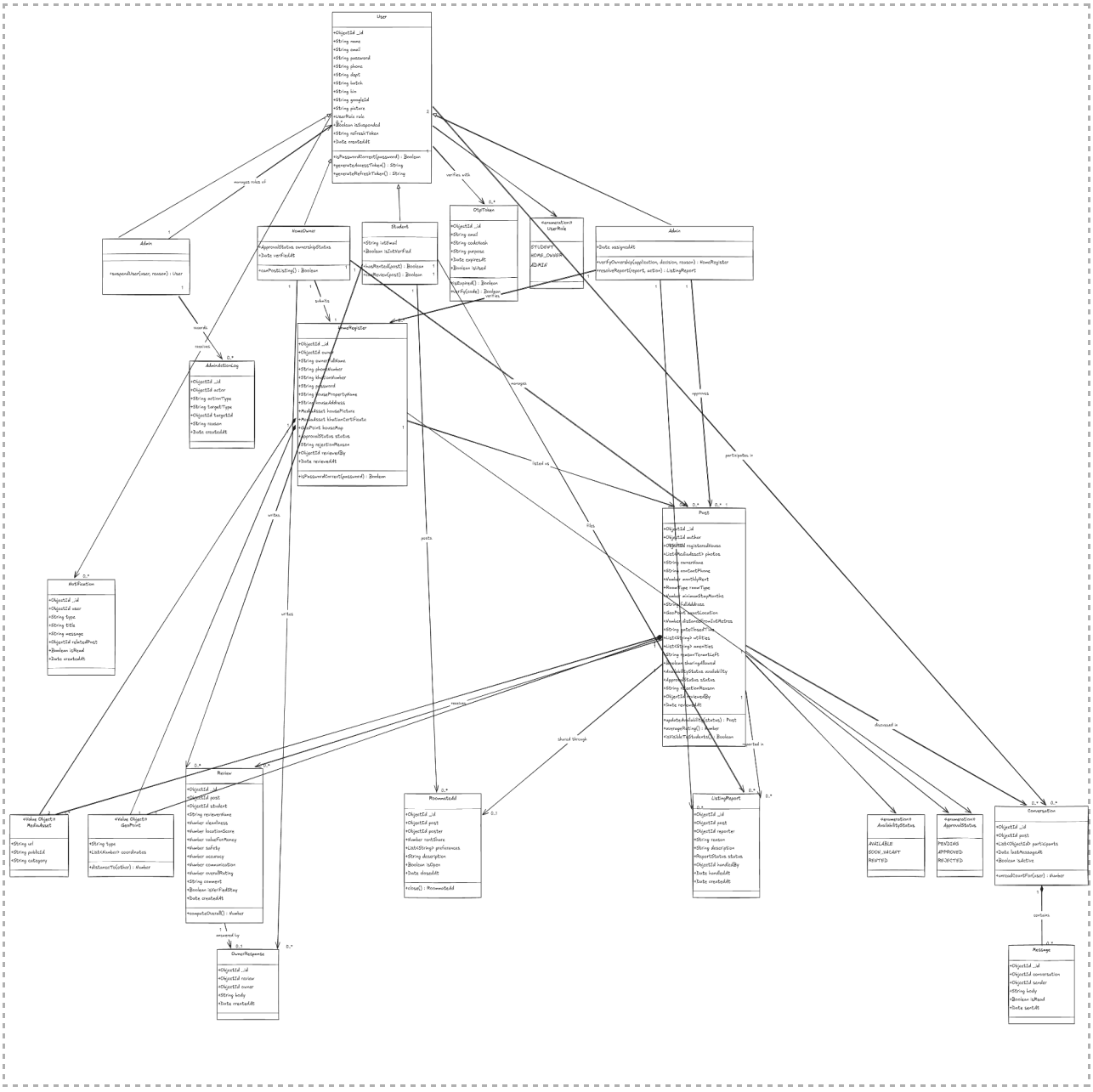


Figure 1: Model Diagram of the Khoj System

Data Flow Diagram

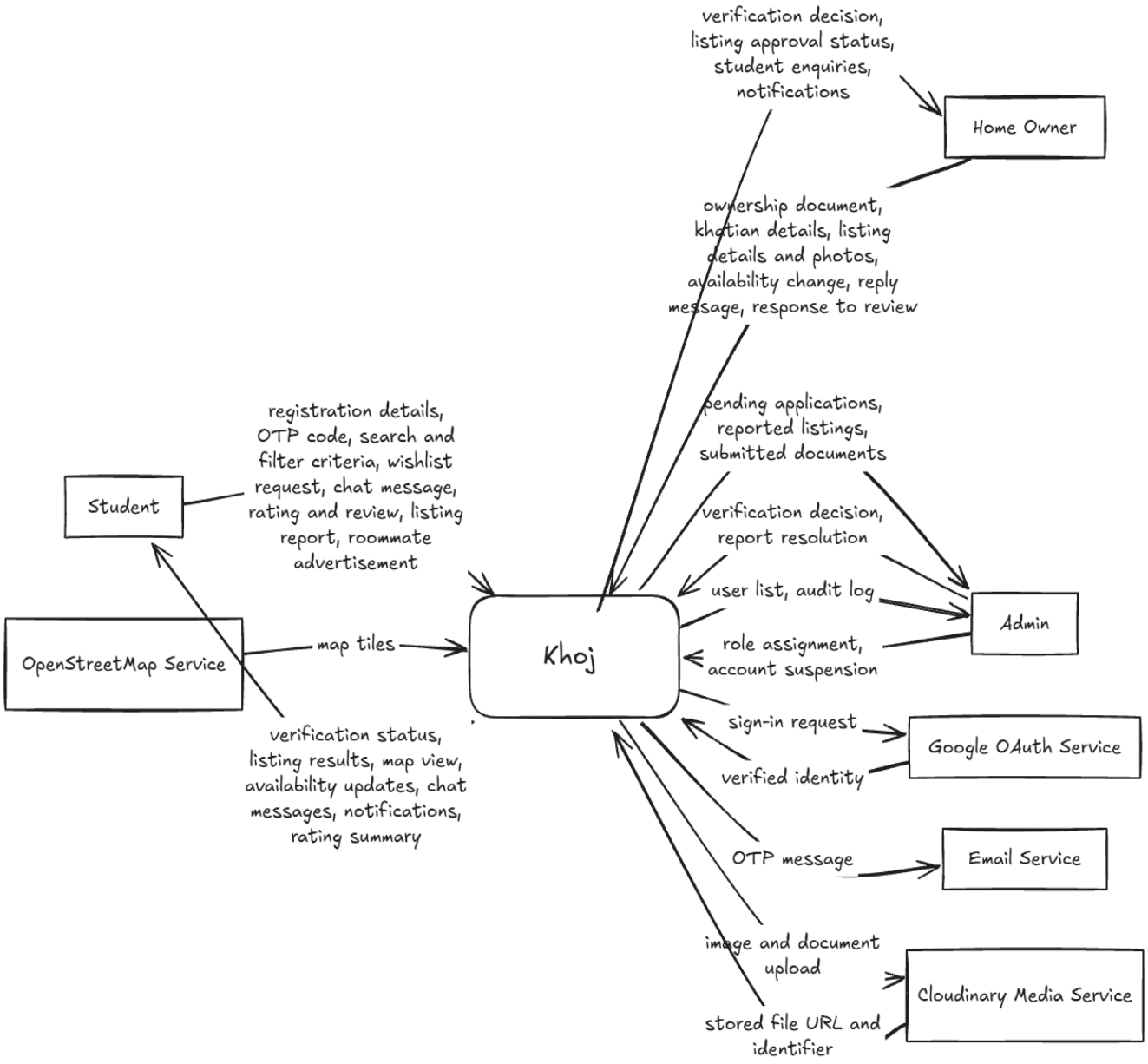


Figure 2: Data Flow Diagram of the Khoj System

1.Explanation of the Model Diagram

1.1 User and Account Models

User is the base model for every account. It stores identity and profile information, the hashed password, suspension flag. It also carries the operations every account needs, whatever its role: checking a supplied password against the stored hash, and issuing the access and refresh tokens that keep a login session alive. Because these sit on the base model, the session mechanism is written once and every role inherits it.

Student extends User with the IUT email address and a flag recording whether that address has been confirmed by OTP. It also declares two operations: whether the student has rented a given room, and whether the student may therefore review it. Together these enforce the rule that only a real tenant can leave feedback.

HomeOwner extends User with the status of the owner's ownership application and the date it was approved. Its single operation reports whether the owner may publish a listing. That answer stays false until the submitted ownership document has been approved.

Admin extends User with the governance operations: suspending accounts.

OtpToken is a single one-time password issued during registration. It gets its own model because it has a lifecycle User does not: it is issued for a stated purpose, it expires at a fixed time, and it can be used once. The model stores only the hash of the code and marks the token used as soon as it verifies, so the same code cannot be replayed.

1.2 Property Verification and Listing models

HomeRegister is the ownership application a homeowner submits. It holds the declared owner details, the khatian number, an image of the khatian certificate, a photograph of the property and its map coordinate. Three approval fields (status, rejection reason, the identifier of the reviewer and the review timestamp) record how verification ended and leave an audit trail behind the decision. No homeowner can publish anything until this record reaches the approved state, so every listing on the platform passes through it.

Post is a room listing. It must reference an approved HomeRegister, so an unverified property cannot reach search results. Its attributes line up with the search requirements: monthly rent, room type, amenity tags and the stored distance from the IUT gate are the three dimensions students filter on, which lets filtering run as a query over indexed fields instead of a calculation after retrieval. Minimum stay duration, gate closing time, utilities and the reason the previous tenant left are there for a different purpose. They are the details that let a student shortlist a room without travelling to see it.

Post carries two status attributes that are easy to confuse. The status attribute is the moderation state: whether the listing has been approved for publication. The availability attribute is the rental state of the room itself, available, soon to be vacant or rented. Students see a listing only once it is approved, and it is the availability value that appears on the listing card and changes without a page reload. A separate sharing allowed flag records whether the owner permits the room to be shared, and a roommate advertisement can only be created against a listing where that flag is set.

1.3 Value Object models

MediaAsset is a file held in the external media service, not in the database. It stores the public URL used to display the file and the storage identifier needed to replace or delete it later. Without that identifier, photographs belonging to a deleted listing would sit in storage forever. Its category attribute records which part of the property a photograph shows: room, bathroom, balcony, kitchen etc. The listing page uses it to group photographs by area so a student can inspect each part of the flat remotely.

GeoPoint stores a location as a GeoJSON point and provides the operation that measures distance between two points. The map view and the distance-from-campus filter both read from this one coordinate, and a geospatial index at the database level answers them without scanning every listing.

Composition attaches both of these to their parents. Neither has an identity or a meaning on its own: a coordinate belongs to exactly one property and an uploaded photograph to exactly one listing, so deleting the parent deletes them too.

1.4 Student Engagement models

Review is a student's rating of a room, written after the stay. It holds six scores from one to five: cleanliness, location, value, safety, accuracy and communication. A written comment can

go with them, and the listing page shows the average as well as the score in each category. Only a student who rented the room can write one.

OwnerResponse is the homeowner's reply to a review. It sits in its own model because a different person writes it, at a different time, and a review can carry at most one reply.

RoommateAd is a post done by a student who occupies a shared room and wants someone to split the rent. It points at the listing concerned, records the share of rent offered and the poster's stated preferences, and closes once a roommate is found. It can only be created against a listing whose owner has permitted sharing.

1.5 Communication and Notification models

Conversation is a chat thread between exactly two participants about one listing.

Message is a single message inside a conversation, recording its sender, its content, the time it was sent and whether it has been read. Composition attaches messages to their thread, since a message means nothing outside the thread that holds it. The same arrangement expresses the requirement that only the two participants can see a conversation.

Notification is an in-app alert addressed to one user. One model covers every alert on the platform, whether a listing has been approved or rejected. The type attribute tells them apart, and the related listing reference lets the alert link straight to the room it concerns.

1.6 Administration Model

AdminActionLog records every governance action taken on the platform: who performed it, what kind of action it was, which record it affected, the stated reason and the time. The requirements call for administrative actions to be logged. Keeping the log in its own model, instead of spreading the fields across the records being acted on, leaves one ordered history that survives even after the affected record is deleted.

1.7 Enumerations

Three enumerations appear in the diagram. Each fixes the complete set of values one attribute may take, so an invalid state cannot be written at all.

Enumeration	Purpose and permitted values
UserRole	The role assigned to an account: STUDENT, HOME_OWNER, ADMIN.
AvailabilityStatus	Whether the room can be taken right now: AVAILABLE, SOON TO BE VACANT, RENTED. This is the first thing a student checks on a listing card.
ApprovalStatus	The moderation state of an ownership application or a listing: PENDING, APPROVED, REJECTED.

1.8 Relationships and Multiplicities

Every relationship drawn in the diagram is listed below with what it means in the system.

Relationship	Type	Multiplicity	Meaning
User - Student / HomeOwner / Admin	Generalisation	Not applicable	Every role inherits identity, credentials and session handling from one base model.
HomeOwner - HomeRegister	Association	1 to 1	One ownership application backs each homeowner account, and that is the record checked before the owner can publish.
HomeRegister - Post	Association	1 to 0..*	A verified property can be listed any number of times, and every listing must point back to one verified property.
HomeOwner - Post	Association	1 to 0..*	An owner manages all of their listings from one dashboard.

Admin - HomeRegister / Post	Association	1 to 0..*	The admin verifies ownership applications and approves listings.
Post - MediaAsset	Composition	1 to 1..*	A listing owns its photographs and takes them with it when deleted.
HomeRegister - MediaAsset	Composition	1 to 2	An application owns the property photograph and the khatian certificate.
Post / HomeRegister - GeoPoint	Composition	1 to 1	Each property and listing owns exactly one map coordinate.
Post - Review	Association	1 to 0..*	A listing collects the reviews written about it.
Student - Review	Association	1 to 0..*	Every review is tied to the student who wrote it, so it can be checked against an actual stay.
Review - OwnerResponse	Association	1 to 0..1	A review may receive at most one reply from the owner.
Post - RoommateAd	Association	1 to 0..1	A shared room may carry one open roommate advertisement.
Post - Conversation	Association	1 to 0..*	Each chat thread concerns one listing, so an owner can tell which room a message is about.
User - Conversation	Association	2 to 0..*	A conversation has exactly two participants and is visible only to them.
Conversation - Message	Composition	1 to 0..*	Messages belong to their thread and are removed with it.
User - Notification	Association	1 to 0..*	Every alert is addressed to exactly one user.

User - OtpToken	Association	1 to 0..*	A user may be issued several one-time passwords over time, each valid once.
Admin - AdminActionLog	Association	1 to 0..*	Every governance action is recorded against the administrator who performed it.

1.9 Traceability to the Functional Requirements

Every model in the diagram comes from a functional requirement in the SRS. Mapping the requirements to the models that implement them makes the design checkable against the specification.

FR	Functional Requirement	Models Involved
FR_1	Students register and log in securely using their IUT email.	User, Student, OtpToken, UserRole
FR_2	Homeowners register by submitting an ownership document, which a admin verifies.	HomeOwner, HomeRegister, MediaAsset, Admin, ApprovalStatus, Notification
FR_3	Students search and filter listings by rent, distance, amenities and room type.	Post, RoomType, GeoPoint
FR_4	Room locations and their distance from IUT are shown on an interactive map.	Post, GeoPoint, RoomType
FR_5	Each room's availability is shown in real time.	Post, AvailabilityStatus, GeoPoint
FR_7	Homeowners create and manage room listings.	HomeOwner, Post, MediaAsset, GeoPoint, RoomType, AvailabilityStatus

FR_8	Real-time in-app chat between students and homeowners.	Conversation, Message, User, Post, Notifiation
FR_9	Students rate and review rooms after their stay; homeowners may respond.	Review, OwnerResponse, Student, HomeOwner, Post
FR_11	Students post roommate advertisements for shared rooms where the owner permits it.	RoommateAd, Student, Post
FR_12	Admins manage users, roles and platform configuration.	Admin, User, UserRole, AdminActionLog

2. Explanation of the Data Flow Diagram

2.1 External Entities and Their Data Flows

Some external entities interact with the system: **three** human roles and services providers. For each one the table gives the data it supplies and the data it receives back.

External Entity	Data Sent to the System	Data Received from the System
Student	Registration details and OTP code, search and filter criteria, chat messages, ratings and reviews, roommate advertisements	Verification status, listing results and map view, availability updates, chat messages, notifications, rating summaries
Home Owner	Ownership document and khatian details, listing details and categorised photographs, availability changes, replies to student messages, responses to reviews	Verification decision, listing approval status, student enquiries, notifications
Admin	Verification decisions with reasons, report resolutions, account suspensions	Pending ownership applications, submitted documents, User list, audit log
Google OAuth Service	Verified user identity	Sign-in request
Email Service	None	One-time password message for delivery
Cloudinary Media Service	Stored file URL and identifier	Image and document uploads
OpenStreetMap Service	Map tiles	Tile requests for the listing map

2.2 Explanation of the Human Entities

The **Student** is the main consumer of the platform. Data flows inward whenever the student registers with an IUT email address and confirms it with a one-time password, applies rent, distance, amenity and room-type criteria to a search, opens a conversation with an owner, reviews a room after a stay, or posts a roommate advertisement. What comes back out is the account verification result, the filtered result list with its map view and each room's distance from the IUT gate, live availability updates, chat messages, in-app notifications and the rating summary on each listing.

The **Home Owner** starts the verification process by supplying the ownership document and khatian details. Once verified, the owner supplies listing details, categorised photographs of the property and changes to a room's availability, plus replies to student messages and responses to reviews. Back out comes the verification decision with its reason if the application was rejected, the approval status of each submitted listing, the enquiries students have sent, and notifications about events affecting the owner's properties.

The **Admin** appears as a separate external entity because admin is a responsibility exercised on the platform, not a use of it. The system supplies pending ownership applications with the documents needed to judge them, and reported listings with the reasons given. The Admin returns approve-or-reject decisions and report resolutions. Each carries a reason, so the outcome can still be explained months later. He also governs the platform and takes no part in the rental process. Account suspensions flow inward. The list of user accounts and the log of administrative actions flow outward. Keeping this entity separate from the Admin follows the requirement that the two responsibilities stay distinct, since the administrator is the one who assigns and revokes the Admin role.

2.3 Explanation of the External Service Entities

The three service providers sit outside the boundary of what the project builds, but the system cannot meet its requirements without them, so the diagram shows them as external entities.

Google OAuth receives a sign-in request and returns a verified identity, letting a student authenticate without the platform ever handling the password.

The **Email Service** receives the one-time password generated during registration and delivers it to the student's IUT address. Nothing comes back from it, which is why only one arrow is drawn.

Cloudinary receives every uploaded image and ownership document and returns the stored file URL along with the identifier needed to replace or delete that file later, keeping photographs and certificates out of the database.

OpenStreetMap answers tile requests from the client with the map tiles drawn behind the listing pins.

Naming these providers has a design purpose beyond documentation. Each is a dependency whose failure or replacement would affect the system. Identifying them at this level lets the implementation keep every one of them behind a single layer, so swapping a provider later does not disturb anything else.

Attachment

- Detailed Software Requirements Specification (User Stories) . Google Sheet
- The diagrams (png). Google Drive